DeployGuard

Railpack Simplification - Proposed Implementation

For confirmation before implementation

Purpose

Document the complete target design, the additional recommendations, what will be deleted, what will remain, and the implementation order. No code changes are implied by this document until the plan is confirmed.

Core decision
Railpack builds the application image. DeployGuard runs that image on AWS. Repository intelligence, stack detection, BuildPlan compilation, topology inference, and Docker template generation are removed from the product path.

1. Executive decision

- DeployGuard becomes an AWS deployment/orchestration product, not a repository-understanding engine.
- Railpack is the only application builder in v1.
- v1 is single-service: one project -> one app root -> one image -> one ECS service -> one port -> one health path.
- The user supplies the few runtime values AWS genuinely needs. DeployGuard does not infer them.
- The old detector/BuildPlan/template architecture is removed, not wrapped behind compatibility adapters.
- Deploy, Redeploy, Rollback, Destroy, ECR, ECS, ALB, optional Postgres, environment variables, secrets, routing, and health verification stay.

Target boundary
Railpack understands source. DeployGuard understands AWS infrastructure and release lifecycle.

2. Final v1 product contract

Required project settings are intentionally small and explicit. Empty required fields block dispatch; there are no silent runtime defaults.

Field	Requirement	Behavior
Repository + branch	Required	Existing project source selection.
App root	Required	Example: . or backend. Exactly one root in v1.
PORT	Required	Injected into ECS runtime; ECS container port and ALB target port use the same value.
Health path	Required	ALB HTTP health check path, such as /health.
Database	Required	none or postgres.
Start command	Optional	Only when Railpack's normal start behavior must be overridden.

```
type ProjectRuntimeSettings = {
  appRoot: string;
  port: number;
  healthPath: string;
  database: 'none' | 'postgres';
  startCommand?: string;
};
```

Naming rule
Do not call this a BuildPlan, canonical plan, topology profile, or analysis result. It is simply user-owned ProjectRuntimeSettings.

3. Final architecture

```
User connects GitHub repository
        |
        v
ProjectRuntimeSettings
(appRoot, port, healthPath, database, optional startCommand)
        |
        v
Checkout exact source revision
        |
        v
Pinned Railpack build
        |
        v
OCI image -> ECR -> immutable image digest
        |
        v
DeployGuard AWS runtime
(ECS + ALB + optional Postgres + env/secrets)
        |
        v
Health / public verification
        |
        v
Deploy / Redeploy / Rollback / Destroy
```

There is no executable stage named Detect Stack, Analyze Repository, BuildPlan, topology compiler, framework resolver, Docker template generation, or detector preflight.

4. Additional recommendations added to the original simplification plan

4.1 Ship single-service v1 only

Do not implement frontend + backend dual roots in this simplification. That recreates topology, routing, and ownership complexity immediately. v1 should support one independently deployable application per DeployGuard project.

```
ONE PROJECT = ONE APP ROOT = ONE IMAGE = ONE ECS SERVICE
```

If a repository contains multiple independently deployable services, the user creates separate DeployGuard projects in v1. Multi-service support can be designed later as an explicit configuration feature, not inference.

4.2 Railpack-only builder; exclude automatic Dockerfile mode

The proposed v1 should have exactly one build authority: Railpack. Do not auto-detect Dockerfiles and do not create a hidden second build path. This prevents DeployGuard from rebuilding repository-intelligence logic around builder selection.

```
builder = railpack
```

If Dockerfile support is added later, it should be an explicit user-selected builder mode, for example railpack | dockerfile. That is outside this v1.

4.3 PORT and HOST are runtime configuration, not build intelligence

Project settings are the sole authority for the application port. DeployGuard passes PORT and HOST to ECS at runtime. The same port is wired to the ECS container definition and ALB target group.

```
ProjectRuntimeSettings.port
  -> ECS env PORT
  -> ECS containerPort
  -> ALB target port

ECS env HOST = 0.0.0.0
ProjectRuntimeSettings.healthPath -> ALB health check path
```

No code path may invent port 3000, 5000, 8080, or health path /. Missing values mean deployment is not dispatched.

4.4 Start command override is explicit

DeployGuard should not infer start commands. Normal builds let Railpack decide. If the user supplies startCommand, the workflow passes it to Railpack using the pinned Railpack version's supported start-command override mechanism.

```
if startCommand is empty:
    railpack build <appRoot>
else:
    railpack build <appRoot> with explicit start-command override
```

The exact CLI syntax is locked to the pinned Railpack release during implementation; it must not float across versions.

4.5 Failure reporting must be simple but accurate

Do not replace the detector with a new “smart” diagnosis layer. Surface the native failure source instead.

Failure source	Product behavior
Railpack/build fails	Show Railpack/build logs.
Image push fails	Show ECR/push error.
ECS task stops	Show ECS stopped reason and task events.
ALB target unhealthy	Show target health reason and configured port/path.
Public check fails	Show HTTP result / timeout.
Database/runtime dependency fails	Show ECS logs/events and existing infrastructure error; do not guess framework causes.

4.6 Separate user settings from release artifact identity

Runtime settings are user configuration. Build output is release identity. Keep those concepts separate.

```
type ReleaseArtifact = {
  sourceSha: string;
  imageUri: string;
  imageDigest: string;
};
```

AWS deployment consumes ProjectRuntimeSettings + ReleaseArtifact + AWS/platform configuration + user env/secrets. It does not consume repository analysis.

4.7 Rollback reuses the previous immutable image

Rollback should switch ECS back to the previously successful image digest. Do not rebuild old source during rollback if the old immutable image already exists.

```
Release A -> image digest A -> healthy
Release B -> image digest B -> healthy
Rollback -> ECS uses digest A again
```

4.8 Add a forbidden-architecture CI guard

After deletion, add a small CI check that fails if old detector/BuildPlan architecture is reintroduced into executable code. This is not a new certification framework; it is a regression guard.

```
Forbidden examples:
StackDetectionService
requireBuildPlan
BUILD_PLAN_REANALYSIS_MESSAGE
MainstreamDetectorResolver
RepoDeployabilityScanner
DockerTemplateEngine
build_plan_base64
generated_dockerfile_base64
topology-detection-v*
```

5. What will be deleted

Deletion should be architectural, not cosmetic. Old code must have no executable readers, writers, workflow inputs, deployment conditions, or UI dependencies.

5.1 Detection area

- backend/src/projects/detection/stack-detection.service.ts
- backend/src/projects/detection/mainstream-detector-resolver.service.ts
- backend/src/projects/detection/framework-detector.ts
- backend/src/projects/detection/python-ast-inspector.ts
- backend/src/projects/detection/repo-deployability-scanner.service.ts
- application-unit discovery / relationship inference / deployment profile / build-provider detector services
- template-matching.service.ts, topology/evidence types, webpack-output helpers
- PHP/Ruby/worker/Celery/Bull or other residual repository-understanding detectors

Repository clone/checkout/commit-resolution behavior is retained but moved into a thin source-workspace service that performs no scanning or detection.

5.2 BuildPlan, topology, templates, and analysis contracts

- build-plan.ts and build-plan-readiness.ts
- analysis-fingerprint / detector-version / repository-analysis refresh concepts when they exist only for detection freshness
- requireBuildPlan(), BUILD_PLAN_REANALYSIS_MESSAGE, detector version pins
- BuildPlan-based deployment contract compilation
- Docker template engine, devops templates, template registry
- detector preflight and generated Dockerfile persistence
- topology/rawProfile/component inventory/relationship inference used to decide deployment
- generated_dockerfile_base64 and build_plan_base64 workflow transport
- candidate evidence matching and BuildPlan byte-for-byte proof checks

5.3 Frontend detection experience

- Detect Stack button and flows
- framework/topology presentation used for deploy readiness
- unsupported framework messages
- detector warnings and “Run Detect Stack again” messaging
- UI dependencies on BuildPlan/topology status

5.4 Detector-specific tests and fixtures

- detector snapshots
- topology-shape tests
- Docker template snapshot tests
- Python AST tests
- mainstream detector test matrices
- generated Dockerfile runtime-smoke tests whose only purpose is validating DeployGuard-generated build instructions

5.5 Database columns from old architecture

Do not create migrations merely to remove old detector/BuildPlan columns during this simplification. Stop reading and writing them. They may remain inert until a later schema squash/cleanup.

> **Hard rule**
> Unused legacy columns may remain temporarily. Legacy architecture may not remain executable.

6. What will stay

Area	Retained capability
Source control	GitHub OAuth/App, installation binding, repository + branch, clone/checkout, exact source revision, workflow dispatch/run tracking.
Build/release transport	Pinned Railpack invocation, ECR push, immutable image digest tracking.
AWS runtime	VPC, ECS cluster/service/task definition, ALB/listener/target group, routing/domain, platform AWS configuration.
Runtime configuration	User environment variables and secrets, PORT/HOST injection, health path.
Database	Existing optional managed PostgreSQL path and secret materialization. No topology ownership expansion.
Lifecycle	Deploy, retry/redeploy, rollback, destroy, release records, failure cleanup/reconciliation.
Verification	ECS stability, ALB target health, public health check, runtime failure visibility.

7. Backend API and persistence implementation

1. Add ProjectRuntimeSettings persistence for appRoot, port, healthPath, database, and optional startCommand.
2. Expose a small settings read/update API used by the project settings form.
3. Validate required settings before every deploy/redeploy. Missing fields return HTTP 400 with the exact missing field names.
4. Remove deploy-time calls to stack detection, BuildPlan readiness, detector freshness, topology validation, and Docker template generation.
5. Dispatch GitHub Actions with source revision + ProjectRuntimeSettings + existing AWS/platform inputs.
6. Persist ReleaseArtifact after a successful build/push so the immutable digest can be deployed and later rolled back.
7. Reconcile workflow/ECS/ALB state without requiring repository analysis.

```
Example validation response:
{
  "error": "PROJECT_CONFIGURATION_INCOMPLETE",
  "missing": ["port", "healthPath"]
}
```

7.1 Settings validation

- appRoot: non-empty relative repository path; reject traversal outside checkout.
- port: integer in valid TCP port range; no default.
- healthPath: non-empty HTTP path beginning with /; no default.
- database: exactly none or postgres.
- startCommand: optional string; absence means Railpack normal behavior.

8. GitHub Actions / Railpack implementation

The reusable workflow becomes the generic build-and-deploy transport. It must not perform repository classification.

1. Checkout the exact requested repository revision.
2. Validate that appRoot exists and that required runtime settings are present.
3. Install/use an explicitly pinned Railpack release. Never float latest.
4. Start/use the required BuildKit backend for that pinned Railpack release.
5. Build the selected appRoot with Railpack. Apply startCommand only if explicitly supplied.
6. Tag/push the image to ECR.
7. Resolve and record the immutable ECR image digest.
8. Run the existing Terraform/ECS deployment path using that digest, the configured port, health path, env/secrets, and optional database outputs.
9. Wait for ECS service stability and ALB target health.
10. Perform a public health check and report the concrete failure source if unsuccessful.

```
Workflow inputs (conceptual):
source_revision
app_root
port
health_path
database
start_command?
image/release identifiers
existing AWS/platform inputs
existing runtime env/secrets references
```

Removed workflow payloads
No build_plan_base64. No generated_dockerfile_base64. No detector fingerprint. No framework/topology evidence. No relationship verification.

9. AWS runtime wiring

9.1 ECS

- Container image = immutable ECR digest from ReleaseArtifact.
- Container port = ProjectRuntimeSettings.port.
- Runtime env includes PORT=<same port> and HOST=0.0.0.0.
- User env/secrets keep the existing runtime injection path.
- No framework-specific container definition logic.

9.2 ALB

- Target group port = ProjectRuntimeSettings.port.
- Health check path = ProjectRuntimeSettings.healthPath.
- Promotion/live readiness is based on actual ECS/ALB health and public verification, not equality with detector output.
- If target health fails, expose AWS target health reason and the configured port/path.

9.3 Routing

Single-service v1 uses the project's normal application route/domain. Do not infer /api or frontend/backend relationships. Any future path routing is explicit configuration and outside v1.

10. Optional PostgreSQL

Keep the existing managed PostgreSQL capability only. Simplification removes component ownership inference because v1 has a single application service.

```
database = 'none'
  -> no managed DB resources

database = 'postgres'
  -> existing managed Postgres provisioning
  -> existing secret/connection materialization
  -> inject connection values into the single ECS application
```

- Do not infer which component owns the database.
- Do not ask Railpack to infer database requirements.
- Do not expand the database product surface while doing this refactor.
- Preserve existing destroy/data-retention policy; do not silently change destructive semantics.

11. Lifecycle semantics after simplification

Operation	Behavior
Deploy	Checkout exact source -> Railpack build -> ECR digest -> ECS/ALB -> healthy/public verification -> mark release live.
Redeploy	Run the same path for the selected/new source. No re-detection and no BuildPlan refresh.
Rollback	Select prior successful ReleaseArtifact and update ECS to its immutable image digest. Do not rebuild old source when prior image exists.
Destroy	Tear down application runtime using existing lifecycle policy. No BuildPlan/topology input is required.
Retry	Retry the failed stage/release path without invoking repository analysis.

12. Frontend experience

After repository selection, replace detector/topology UI with one small settings form.

```
Application folder     [ .               ]
Port                   [ 5000            ]
Health check path      [ /health         ]
Database               [ None | Postgres ]

Advanced
Start command          [ optional         ]

                         [ Deploy ]
```

- Deploy button disabled while required fields are empty, or backend rejects dispatch with exact missing fields.
- No Detect Stack button.
- No framework badge required for deployment readiness.
- No topology panel.
- No unsupported-framework state.
- No detector warnings or reanalysis messaging.
- Build/deploy failure view links directly to the Railpack/GitHub Actions/AWS failure evidence.

13. Replacement test and certification strategy

DeployGuard should test its own boundary, not re-test the universe of frameworks that Railpack supports.

- Explicit appRoot reaches Railpack unchanged.
- Railpack failure propagates with useful logs.
- Successful image is pushed to ECR and immutable digest is recorded.
- Configured port reaches ECS env, ECS containerPort, and ALB target port identically.
- Configured healthPath reaches ALB identically.
- User env/secrets reach the running ECS task.
- Optional PostgreSQL connection reaches the single application.
- Healthy deployment becomes live and public URL responds.
- Failed candidate does not silently replace the last healthy release.
- Redeploy works without detection.
- Rollback switches to the previous immutable digest.
- Destroy removes the expected application resources and preserves/deletes DB according to existing policy.
- Old detector/BuildPlan forbidden-architecture CI check passes.

13.1 Smoke applications

Use at least two generic smoke repositories only to prove the integration boundary: one Node application and one Python application. They are not special-cased in DeployGuard code.

14. Implementation order

Do the migration in slices so there is always one understandable executable path. Do not mix detector fixes with the simplification.

1. Introduce ProjectRuntimeSettings persistence/API and frontend form. Required fields must block deploy when missing.
2. Add the pinned Railpack build path to the reusable workflow and produce an immutable ECR image digest.
3. Wire ProjectRuntimeSettings.port and healthPath directly into ECS/ALB runtime configuration.
4. Wire optional startCommand into Railpack only when supplied.
5. Change deploy/redeploy to use settings + ReleaseArtifact and stop calling stack detection / BuildPlan readiness.
6. Make rollback use the prior successful image digest.
7. Remove Detect Stack UI and all detector-readiness presentation.
8. Delete detector services and move clone/checkout/commit SHA behavior into a thin source service.
9. Delete BuildPlan/topology/template/preflight code and strip remaining imports/consumers.
10. Delete detector-specific tests/scripts/fixtures and replace with boundary tests.
11. Add forbidden-architecture CI guard.
12. Run Node + Python end-to-end AWS smoke: deploy, public health, redeploy, rollback, destroy.

Migration rule

Do not create a compatibility layer that lets old detector/BuildPlan output feed the new Railpack path. The new path must become authoritative, then the old path is removed.

15. Explicit non-goals for this v1

- Framework lists or compatibility matrices inside DeployGuard
- Package-manager inference
- Python AST or JavaScript source analysis
- Static-vs-SSR inference
- Worker/Celery/Bull detection or blocking
- Angular/Svelte/Next/Django special-case proof logic
- Automatic Dockerfile selection
- Multi-service frontend/backend topology
- Inferred /api routing
- BuildPlan or detector freshness/reanalysis
- New database feature expansion
- A new “smart failure analyzer” that guesses application causes

16. Definition of success

- New project: user selects repo, enters app root + PORT + health path + database choice, and deploys.
- Old project: user fills the new required settings once; no re-detection is required.
- Railpack build failure is visible as build failure with logs.
- ECS/ALB failure is visible using native task/target-health evidence.
- A healthy image is deployed by immutable digest.
- Redeploy uses the same simple path.
- Rollback returns ECS to the previous successful digest.
- Destroy works without BuildPlan/topology.
- No executable stack-detection or BuildPlan dependency remains.
- No code defaults PORT or health path.
- No Dockerfile template is generated by DeployGuard.
- Forbidden-architecture CI guard prevents accidental reintroduction.

Product in one sentence
Railpack in. Our scanner out. AWS stays. User provides app root, port, health path, and database choice. DeployGuard builds, ships, runs, verifies, rolls back, and destroys the image.

17. Confirmation checklist before implementation

Approve or change these decisions before code is modified:

☐ v1 is single-service only.

☐ Railpack is the only v1 builder.

☐ No automatic Dockerfile mode in v1.

☐ No framework/runtime/package-manager detection in DeployGuard.

☐ appRoot, port, healthPath, database are required user settings.

☐ No default PORT and no default health path.

☐ HOST=0.0.0.0 is platform runtime configuration.

☐ startCommand is optional and only an explicit Railpack override.

☐ ProjectRuntimeSettings replaces BuildPlan as the deployment input model.

☐ ReleaseArtifact records sourceSha + imageUri + imageDigest.

☐ Deploy/redeploy never call detector/BuildPlan.

☐ Rollback reuses a prior immutable image digest.

☐ Detector/BuildPlan/template/preflight code is deleted, not adapted.

☐ Old detector DB columns may remain temporarily but are never read/written.

☐ Existing AWS, env/secrets, optional Postgres, lifecycle, health, and cleanup paths remain.

☐ Failure reporting surfaces Railpack/GitHub Actions/ECS/ALB evidence without new repo intelligence.

☐ A forbidden-architecture CI check is added after deletion.

☐ Final smoke certification is Node + Python on real AWS lifecycle.

18. Implementation boundary

This proposal describes the target implementation. It does not authorize hidden additions beyond the scope above. During implementation, any discovered dependency should be classified as one of:

- DELETE - exists only for repository detection/BuildPlan/template intelligence.
- KEEP - required to ship/run/reconcile an image on AWS.
- REWIRE - required AWS/lifecycle code currently consumes BuildPlan and must instead consume ProjectRuntimeSettings or ReleaseArtifact.
- DEFER - useful capability but outside single-service Railpack v1.

Final implementation constraint
No old architecture should survive merely because deletion is inconvenient. If a kept AWS feature currently depends on BuildPlan, rewire that feature to explicit user settings or release artifact identity, then delete the BuildPlan dependency.

Status: PROPOSED - awaiting confirmation before implementation.